

Executive Summary

This design augments OpenChoreo's multi-plane architecture [13†L269-L278] with a GitOps pipeline triggered by Backstage. Developers use a Backstage/Scaffolder portal to input intent (e.g. new service parameters). The Scaffolder then invokes the OpenChoreo CLI (`occ`) to generate Kubernetes manifests (Workload, ComponentRelease, ReleaseBinding) and uses built-in actions to commit them to a Git repo and open a PR [64†L259-L267] [51†L16-L24] [52†L14-L18]. Upon PR merge, a GitOps operator (FluxCD or ArgoCD) detects the change and applies it to the cluster, where OpenChoreo's Control Plane reconciles the high-level CRDs into real Deployments, Services, etc [64†L259-L267] [13†L269-L278]. This report details the architecture (ASCII and Mermaid diagrams), a 6-month rollout plan, repo layout options, example Scaffolder code and YAML, Git provider integration patterns, and FluxCD/ArgoCD tradeoffs.

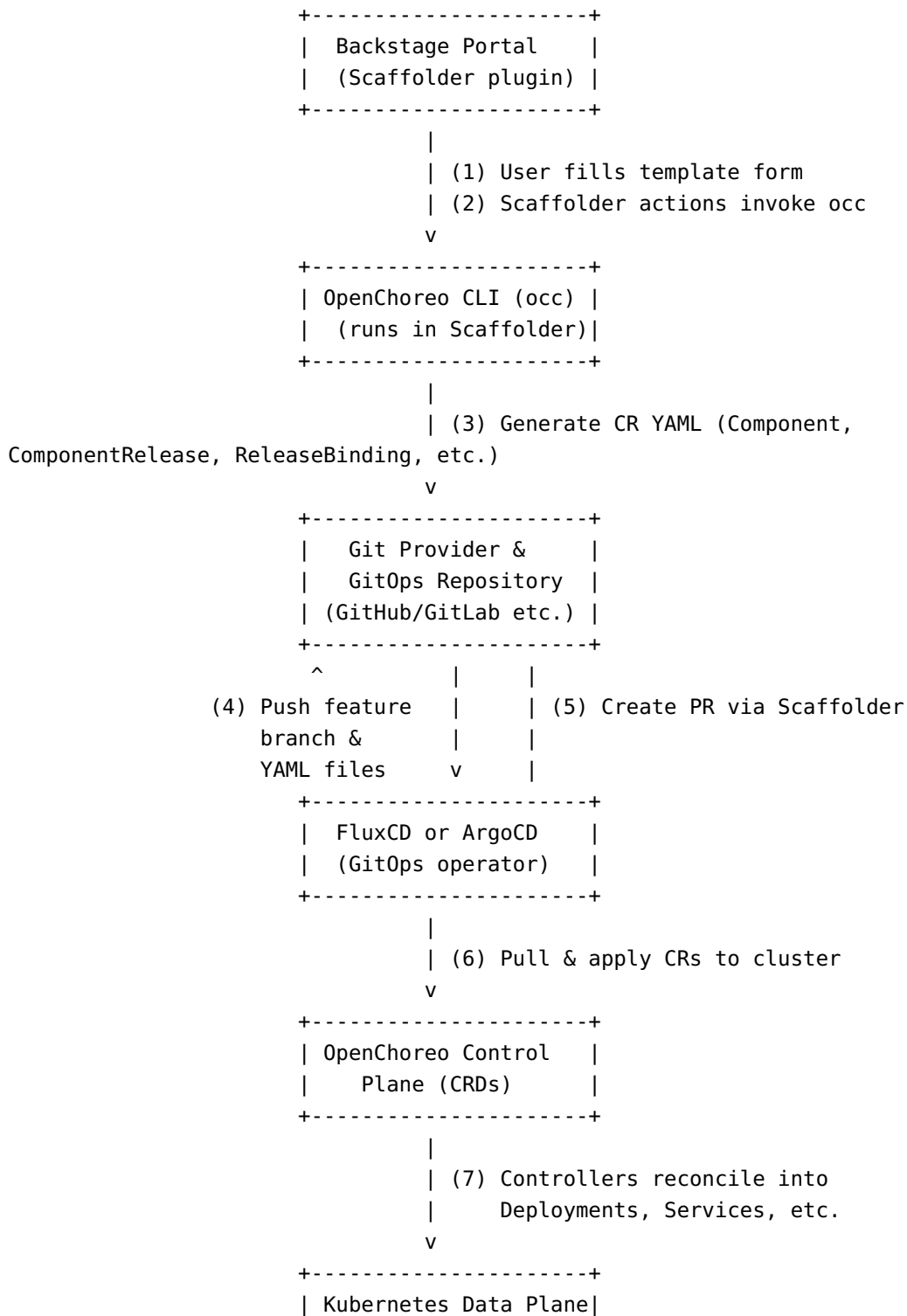
Goals and Success Criteria

- **Self-Service Velocity:** Developers can create or update components via Backstage forms, without manual YAML edits. Templates and the `occ` CLI handle manifest generation, while GitOps automates deployment [64†L259-L267]. Success means "git push" and "merge PR" are the only manual steps.
- **Alignment with OpenChoreo:** All OpenChoreo abstractions (ComponentTypes, Traits, Workflows, etc.) continue to work. The pipeline must generate the same CRDs as a manual process (e.g. `occ component scaffold` or `occ component deploy`). Outputs like `ComponentRelease` and `ReleaseBinding` must function unchanged [64†L259-L267].
- **Robustness and Safety:** The system correctly handles concurrent changes (conflict detection), enforces least-privilege credentials, and provides observability. Errors (syntax, merge conflicts, failed syncs) are clearly reported. Success criteria include passing automated tests (unit and e2e) and a smooth demo scenario (see *Acceptance Tests*).
- **Documentation and Demo:** All workflows must be documented; an end-to-end demo (e.g. recorded or live) shows a developer creating a component in Backstage, opening/merging a PR, and seeing it deployed.

Assumptions and Constraints

- **Constraints:** One developer, 6-month schedule. Starting baseline: OpenChoreo v1.x on a local k3d cluster, with Argo Workflows already deployed (for CI builds). No fixed cloud or Git provider is mandated ("no specific constraint"); we will use GitHub/GitLab as examples.
- **Underspecified Items (Assumptions):** We assume HTTPS-based Git provider (GitHub or GitLab) with API tokens available. We assume existing platform definitions (ComponentTypes/Traits) are already configured on the cluster. Network/domain for Backstage UI is in place. For secrets, we assume a Kubernetes secrets store (e.g. External Secrets/ClusterSecretStore).
- **Environments:** Typically "dev" and "prod" environments exist. We assume a single-cluster demo (k3d) with one ArgoCD/Flux instance; multi-cluster support can be added later.

Architecture Overview



```
| (Actual workloads) |
+-----+
```

This architecture uses Backstage as the *experience plane*, consistent with OpenChoreo's design [13†L269-L278] [39†L117-L124] . The Scaffolder plugin runs a sequence of actions: it fetches a template, calls the `occ` CLI (via a script action) to populate a manifest, then uses Git actions to push to the Git provider and open a PR [51†L16-L24] [52†L14-L18] . The GitOps operator (Flux or ArgoCD) in the cluster continuously watches the designated repo/branch; on merge it detects new CRDs and applies them. OpenChoreo's control plane then sees the new `ComponentRelease` or `ReleaseBinding` resources and creates the underlying workload (the *data plane*) [64†L259-L267] [13†L269-L278] . The Observability plane (Prometheus, logs, tracing) captures metrics from Backstage, the CLI jobs, and controllers [68†L7-L10] .

```
graph LR
  subgraph Experience Plane
    B(Backstage Portal)\n(Scaffolder) --> OccCLI[OpenChoreo CLI (occ)]
    OccCLI --> GitDeploy[YAML manifests generated]
    GitDeploy --> GitPublish[Publish to GitHub/GitLab]
    GitPublish --> GitRepo((GitOps Repo))
  end
  subgraph Git Provider
    GitRepo --> PR[Pull Request]
  end
  subgraph GitOps Plane
    PR -->|merge| Flux[FluxCD / Argo CD]
    Flux --> ControlPlane[OpenChoreo Control Plane (CRDs)]
    ControlPlane --> K8s[Kubernetes Resources]
  end
```

Key components and interactions: Backstage's Scaffolder (experience plane) interacts with the Git provider via plugin actions [51†L16-L24] [52†L14-L18] . The `occ` CLI (packaged with the Scaffolder or invoked inside a container) generates OpenChoreo CRs from user input. A **GitOps repo layout resolver** (in code or config) determines where to place files (see below). PR creation is handled by the Scaffolder's GitHub/GitLab actions, requiring credentials. When a PR is merged, FluxCD/Argo CD (cluster controllers) detect the commit and apply manifests. OpenChoreo controllers (control plane) then see the new CRs (Component, Project, Releases, etc.) and roll out actual workloads.

Security and Credential Management

- **Least-Privilege Git Credentials:** Store Git provider tokens in Kubernetes secrets or Backstage config, not in code. For example, use a GitHub App or personal token with **only** repo write and PR-create permissions. OpenChoreo workflows store tokens in a `ClusterSecretStore` (e.g. `gitops-token`) [64†L269-L276] . The Scaffolder Git actions take a token as input [51†L16-L24] [52†L14-L18] , so we must ensure it's injected securely (e.g. via environment or Vault). For GitLab, a similar setup is used (personal access token or GitLab app).

- **RBAC:** On Kubernetes, give Flux/Argo service accounts only the needed permissions (namespaces, CRDs). The OpenChoreo controllers already run with the cluster-admin role or equivalent (pre-existing). The `occ` CLI in scaffolder needs no cluster privileges (it outputs YAML only).
- **Secret Management:** Never commit secrets in Git. Use tools like Sealed Secrets or External Secrets for any secrets referenced in manifests [\[45†L393-L399\]](#) . In GitOps repos, credentials (e.g. image registry creds) should be externalized (Flux *SOPS*, ArgoCD Vault plugin, etc.) [\[45†L393-L399\]](#) .

Conflict Detection and Concurrent PR Handling

With multiple developers (or automated workflows), two PRs might target the same resources. Our strategy: before opening a new PR, the Scaffolder action can *query the Git provider for open PRs* in the target repo and branch. If an existing PR touches the same files or components, the action can either alert the user or merge changes into that PR. For example, the GitHub action `publish:github:pull-request` can be run with `update=true` to update an existing PR branch [\[52†L14-L18\]](#) . Alternatively, we can implement a conflict check step (script) that inspects `git diff` against `main` . Mitigation: require manual review of concurrent changes, or queue PRs. Acceptance test: attempt two scaffolds of the same component and verify the system warns or queues.

Dependency Resolution (ComponentTypes and Traits)

OpenChoreo allows defining ComponentTypes/Traits at cluster (platform) or namespace (project) level. Our design must handle both “global” and “local” definitions. The Scaffolder will accept a parameter like `typeKind` (ClusterComponentType vs. ComponentType) and `typeName` . Internally, it can *fetch the CRD schema* from the cluster (via `kubectl get componenttype ...`) to prompt for parameters. If the type is cluster-scoped, use `--clustercomponenttype` ; if project-scoped, use `--componenttype` . Hybrid resolution means supporting both: e.g. if a namespace defines a special override, use that; else fall back to cluster type. The `occ` CLI already handles this (it looks for the named type in cluster or namespace). We simply need to feed it the correct kind/name. (Platform docs: “ComponentTypes and Traits...let platform teams define golden paths” [\[13†L308-L312\]](#) .) We should plan to validate trait names similarly.

Observability and Telemetry

Every step should emit metrics and logs. The Observability Plane in OpenChoreo (Prometheus, etc.) will collect:

- **Scaffolder metrics:** count of scaffolds, PRs created, errors. We can integrate OpenTelemetry in custom scaffolder code.
- **Controller metrics:** OpenChoreo controllers already expose Prometheus metrics (e.g. reconciliation latency).
- **GitOps operator metrics:** Flux/Argo export sync health and counts.
- **Logging:** Backstage and workflows should log to a central logger (Elastic/CloudWatch/etc.). Trace IDs could link the Backstage request through the GitOps pipeline.

- **Dashboards:** Build dashboards showing number of deployed components, pipeline failures, etc. (OpenChoreo’s observability module [\[68†L7-L10\]](#)).

Testing Strategy

- **Unit Tests:** For any new scaffolder actions or resolvers, write unit tests (Jest/TypeScript for Backstage actions, Go tests for controllers).
- **Integration Tests:** Use a KinD/k3d cluster to deploy the full system (Backstage + OpenChoreo + Flux or Argo) in a GitOps mode. Then simulate user flows: use the Scaffolder to create a component, check the PR is created, merge it, and verify the resource appears in the cluster (Kubernetes Deployment running).
- **Acceptance Tests:** Automate a “happy path” scenario and edge cases (invalid input, concurrent PR). For example, a test that runs `occ component scaffold` on sample data and diffs the output against expected YAML templates. Another test can simulate PR conflicts by attempting two simultaneous deploys of the same name and asserting the system flags a conflict.
- **End-to-End Demo:** Document a manual e2e test: e.g. developer creates a “guestbook” service in Backstage, scaffold runs, a PR is opened, reviewer merges it, and application is live. This demo should be part of the final acceptance criteria.

CI/CD for Scaffolder and Controllers

- **Scaffolder (Backstage plugin) CI:** The scaffolder templates and any custom actions should live in a GitHub repo. CI (GitHub Actions) will run linting, unit tests, and build (compile TS, publish a Docker image or npm package). Use CD pipelines to deploy updates to the Backstage dev environment (containerized).
- **OpenChoreo Controllers:** These are Go projects in the OpenChoreo repo. CI will run `go test` and linters. Releases should produce container images published to a registry (e.g. GHCR or DockerHub). We then update the cluster with Helm or kubectl apply.
- **Flux/Argo Bootstrap:** We should bootstrap the GitOps repo with Flux/Argo manifests (maybe commit via pipeline or manual). Once bootstrapped, their reconciliation is automatic.

Documentation and Demo Plan

- **Docs:** Update OpenChoreo docs with “PR-Driven GitOps” section. Include Backstage screenshots, Scaffolder YAML examples, and workflow diagrams. Document assumptions, how to configure tokens, and the repo layout.
- **Tutorial:** Write a step-by-step tutorial (on [openchoreo.dev](#) or a repo README) for “Deploying via Backstage”.
- **Demo Script:** Prepare a demo script and recording showing (a) setting up a new project in Backstage, (b) scaffolding a component, (c) reviewing and merging the PR, (d) seeing the service appear. Highlight observability screens (logs, metrics) at the end.
- **Training:** Possibly a brief walkthrough for the platform team on how to extend templates or traits.

(A) Architecture Diagram

ASCII Diagram:

```
[Backstage Portal (Experience Plane)]
|
v
[Scaffolder Plugin] -- (invokes occ) --> [occ CLI Container]
|
v
+-----+
| Git Provider (GitHub/GitLab) |
| - GitOps Repository          |
| - On PR merge: FluxCD/ArgoCD |
+-----+
|
v
[FluxCD/ArgoCD (GitOps operator)]
|
v
[OpenChoreo Control Plane (CRD controllers)]
|
v
[Kubernetes Data Plane (Deployments, Services, etc.)]
```

Mermaid Diagram:

```
graph LR
    subgraph Backstage_Experience
        A[Backstage UI\n(Experience Plane)] --> B[Scaffolder Plugin]
        B --> C["Template \n& occ CLI"]
        C --> D[Generated CRD YAML]
        D --> E[Git Provider (Repo)]
    end
    subgraph GitOps_Plane
        E -- "PR merge" --> F["FluxCD / ArgoCD"]
        F --> G[OpenChoreo Control Plane]
        G --> H[Kubernetes Data Plane]
    end
    subgraph Observability
        A -. "Metrics/Logs" .-> I["Prometheus/Grafana"]
        G -. "Metrics/Logs" .-> I
    end
```

This diagram shows the flow from the Backstage portal through the scaffolder and occ, into the GitOps repo, and finally into the cluster via Flux/Argo and the OpenChoreo control plane. Metrics flow into the observability stack.

(B) 6-Month Implementation Roadmap

Month	Milestone & Deliverables	Major Tasks (Effort)	Risks & Mitigations	Acceptance Tests
Month 1: Planning & Environment Setup (4 weeks)	- Detailed design document - Local lab: k3d cluster with OpenChoreo v1.x installed [64†L259-L267] and Backstage dev environment - Evaluate Git provider (e.g. GitHub) and CI pipelines	- Review Project 702 specs and OpenChoreo docs (0.5w) - Set up local cluster with OpenChoreo and Argo Workflows (1w) - Stand up Backstage with Scaffolder plugin (1w) - Install FluxCD/ArgoCD on cluster for later (0.5w) - Document setup (1w)	- Risk: Ambiguity in requirements. <i>Mitigation:</i> Clarify assumptions now, and iterate prototype early. - Risk: Environment issues (e.g. Backstage complexity). <i>Mitigation:</i> Use official quickstart guides [39†L117-L124] [64†L259-L267] .	- Can successfully login to Backstage portal and see OpenChoreo catalog. - occ CLI runs in dev (e.g. occ version outputs). - FluxCD/ArgoCD installed and reachable.

Month	Milestone & Deliverables	Major Tasks (Effort)	Risks & Mitigations	Acceptance Tests
Month 2: Scaffold & occ Integration (4 weeks)	- Scaffolder templates defined for at least one ComponentType - <code>occ</code> CLI integration tested - Scaffolder action to run <code>occ</code> to generate YAML	- Create Backstage template repository (service template with <code>scaffolder/*</code> folder) (0.5w) - Develop template scaffolder (questions for componentName, type) (0.5w) - Invoke <code>occ</code> via a script action (0.5w); ensure output YAML captured - Write unit tests for scaffolder actions (1w) - Demo: run scaffolder in portal, verify YAML output	- Risk: “run:command” or CLI integration complexity. <i>Mitigation:</i> Use existing NodeJS action or a Docker-based action as needed. Backstage has examples <code>【51†L16-L24】</code> . - Risk: Parameter mismatch. <i>Mitigation:</i> Validate against OpenChoreo schemas beforehand.	- Scaffolder run produces YAML files (Component, ComponentRelease, etc.). - Example: <code>occ component scaffold myservice</code> yields correct CRD (compare to docs).

Month	Milestone & Deliverables	Major Tasks (Effort)	Risks & Mitigations	Acceptance Tests
Month 3: GitOps Repo & PR Automation (4 weeks)	- GitOps repository scaffolded (monorepo or multi-repo) - Scaffolder actions to push branch and create PR 【51†L16-L24】 【52†L14-L18】 - GitHub/GitLab integration configured	- Decide repo layout (see next section) (0.5w) - Create GitOps repo with sample structure (0.5w) - Add Scaffolder “publish:git” and “create PR” steps (0.5w) - Configure Git provider tokens and test connection (1w) - Test flow: scaffold -> auto-Push -> PR created (1w)	- Risk: API rate limits or permission errors. <i>Mitigation:</i> Use machine user or app with minimal scopes; retry logic. - Risk: Merge conflicts on initial PR. <i>Mitigation:</i> Always work on feature branch from known base; use dry-run to detect conflict.	- On scaffolding, a feature branch is created in GitOps repo with correct YAML. - A PR is automatically opened against <code>main</code> (or target branch). - Verify PR contains expected files (see sample YAML below).

Month	Milestone & Deliverables	Major Tasks (Effort)	Risks & Mitigations	Acceptance Tests
Month 4: GitOps Operator Sync (4 weeks)	- FluxCD or ArgoCD sync configured to watch the repo - On PR merge, OpenChoreo components deploy automatically 【64†L259-L267】 - End-to-end pipeline works for a sample component	- Install ArgoCD (or Flux) in cluster if not done (0.5w) - Bootstrap GitOps repo for operator (e.g. <code>argocd-apps</code> or <code>flux bootstrap</code>) (0.5w) - Define Application/ ApplicationSet (Argo) or Kustomization (Flux) to point at repo (1w) - Merge test PRs and verify resources created (1w) - Handle first ProjectRelease/ ReleaseBinding flow in cluster (0.5w) - E2E test in lab environment (0.5w)	- Risk: Operator misconfiguration (sync policy, creds). <i>Mitigation:</i> Use simple one-app test first; enable “prune” carefully after tests. - Risk: YAML errors cause sync failures. <i>Mitigation:</i> Lint YAML in CI, use dry-run apply in test.	- Merging a PR leads to Flux/Argo detecting changes and applying them. - The new component (e.g. a Deployment) appears in cluster, and <code>kubectl get components</code> shows status READY.

Month	Milestone & Deliverables	Major Tasks (Effort)	Risks & Mitigations	Acceptance Tests
Month 5: Advanced Features & Refinement (4 weeks)	- Conflict detection logic implemented
- Support for multiple environments (dev/prod), promotions
- Hybrid type/trait resolution ensured
- Observability dashboards configured	- Implement PR conflict check in scaffolder (1w)
- Support “promote” by creating ReleaseBinding updates (0.5w)
- Enable multi-environment: scaffold should create bindings per env (0.5w)
- Add ability to set parameter overrides on promotion (0.5w)
- Create metrics (e.g. Prometheus) for scaffolder actions (1w)
- Document security model (0.5w)	- Risk: Feature creep (trying to do too much). <i>Mitigation:</i> Focus on core flow; schedule “stretch” items as time allows.
- Risk: Complex merge scenarios. <i>Mitigation:</i> Limit scope to linear workflows initially (e.g. one PR at a time).	- Two simultaneous scaffolds detect conflict and one is flagged or serialized.
- Promote action (via CLI or Backstage) advances ReleaseBinding to new release; verify cluster reflects promoted version.
- Metrics dashboard shows number of scaffolds and syncs.
Month 6: Testing, Documentation, and Demo (4 weeks)	- Full test suite passing
- Complete documentation (user guide, API, architecture) 【64†L259-L267】 【45†L393-L399】
- Demo/ presentation completed	- Write additional unit tests for edge cases (1w)
- Perform load test or continuous usage test (0.5w)
- Finalize user and dev documentation (1w)
- Prepare demo scenario and video (1w)
- Buffer / bugfix (0.5w)	- Risk: Insufficient test coverage. <i>Mitigation:</i> Peer review test plan, use checklists.
- Risk: Schedule slip. <i>Mitigation:</i> Leave 1 week buffer for last-minute issues.	- Automated tests confirm no regressions.
- Documentation covers setup and workflow (excerpts or diagrams cited) 【51†L16-L24】 【64†L259-L267】 .
- Demo run-through (recording or live) succeeds without errors.

(C) Repository Layout Options

Option	Structure Example	Pros	Cons
Monorepo (single)	/cluster-config/	- Centralized single repo for all apps/environments [55†L247-L256] . - Simplifies branch strategy (e.g. one <code>main</code> for all). - ArgoCD style: base/ overlays per env [55†L247-L256] .	- Repo grows large over time. - Potential for merge conflicts across teams. - Requires disciplined folder structure (e.g. <code>apps/</code> and <code>environments/</code>).
Multi-Repo: Per App	svc-web , svc-api , ... (each repo contains its K8s manifests)	- Teams manage only their repo. - Scalable: no contention on one repo. - FluxCD can bootstrap each repo independently.	- Harder to manage cross-cutting policies (needs many Flux/Argo applications). - More overhead for platform team (must set up each new repo).
Multi-Repo: Per Env	cluster-dev , cluster-prod (apps inside)	- Strong environment isolation (each env in own repo). - Clear separation (prod changes not in dev repo).	- Duplicated app configs across env repos (drift risk). - Promotion requires synchronizing changes between repos.
Hybrid (single with submodules)	One repo with git submodules for components	- Teams develop in submodules (their own namespace). - Central repo ties everything together.	- Complex setup (submodule management burdens). - Potential confusion; rarely used in GitOps context.

Recommendation: Start with a **monorepo** layout similar to the ArgoCD example [55†L247-L256] : e.g.

```
gitops-repo/
├─ components/
│   └─ my-service/
│       ├── base/
│       └─ overlays/dev/, overlays/prod/
├─ argocd-apps/
│   ├── my-service-dev.yaml
│   └─ my-service-prod.yaml
└─ README.md
```

This allows a single PR to modify multiple services if needed and leverages ArgoCD's base/overlay pattern. Table-driven promotion (dev→prod) can be handled by updating overlays or using ApplicationSets. The

monorepo simplifies onboarding for a 1-person team, given the 6-month horizon. In the future, a multi-repo strategy could be considered if scaling to many teams.

(D) Sample Scaffolder Action and occ YAML

Scaffolder Template (pseudocode):

```
apiVersion: backstage.io/v1alpha1
kind: Template
metadata:
  name: create-component
  title: "Create OpenChoreo Component"
  description: "Scaffold a new component and GitOps PR"
spec:
  parameters:
    - title: Component ID
      type: string
      name: componentId
    - title: Component Type
      type: string
      name: componentType
    - title: Environment
      type: string
      name: environment
      enum: [dev, staging, prod]
  steps:
    - id: fetch-template
      action: fetch:template
      input:
        url: "https://github.com/org/infra-templates/blob/main/openchoreo-
component"
        targetPath: "."
        values:
          componentId: ${ parameters.componentId }
          componentType: ${ parameters.componentType }
    - id: generate-manifests
      action: run:command
      input:
        command: |
          # Run occ CLI to generate component & release
          occ component scaffold ${ parameters.componentId } \
            --type ${ parameters.componentType } \
            --namespace ${ parameters.componentId } \
            --output ./
          # (This outputs component.yaml and release-binding templates)
    - id: git-push
```

```

    action: github:repo:push
    input:
      repoUrl: github.com?owner=example&repo=gitops-config
      branch: feature/${{ parameters.componentId }}
      commitMessage: "scaffold: add ${{ parameters.componentId }} component"
      sourcePath: .
  - id: create-pr
    action: publish:github:pull-request
    input:
      repoUrl: github.com?repo=gitops-config&owner=example
      branchName: feature/${{ parameters.componentId }}
      targetBranchName: main
      title: "Add component ${{ parameters.componentId }}"
      description: "Scaffolded new component via Backstage"
      reviewers:
        - platform-lead

```

This template does: (1) fetch a Jinja/Handlebars template, (2) run the `occ` CLI to create YAML, (3) push to a new feature branch, and (4) open a PR. The actions `github:repo:push` and `publish:github:pull-request` come from Backstage Scaffolder plugins [\[51†L16-L24\]](#) [\[52†L14-L18\]](#) .

Generated `occ` YAML (sample): Suppose `parameters.componentId=webapp` , `componentType=ClusterComponentType:service` , and `environment=dev` . The `occ` commands would produce CRDs like:

```

# components/webapp.yaml
apiVersion: openchoreo.dev/v1alpha1
kind: Component
metadata:
  name: webapp
spec:
  type:
    kind: ClusterComponentType
    name: service
  parameters:
    image: "dockerhub.io/myorg/webapp:latest"
    port: 8080

---
apiVersion: openchoreo.dev/v1alpha1
kind: ComponentRelease
metadata:
  name: webapp-<hash>
spec:
  component: webapp
  parameters: {}

```

```

---
apiVersion: openchoreo.dev/v1alpha1
kind: ReleaseBinding
metadata:
  name: webapp-dev
spec:
  componentRelease: webapp-<hash>
  environment: dev

```

In practice, `occ` would compute `<hash>` from the component spec. The scaffolder then commits these into the appropriate directory in the GitOps repo (e.g. under `components/webapp/overlays/dev/`).

(E) GitHub/GitLab Integration & Credentials

- **Backstage Plugin Actions:** Backstage provides ready-made actions for GitHub and GitLab (plugins `@backstage/plugin-scaffolder-backend-module-github` and `...-gitlab`) to create branches and PRs [51†L16-L24] [52†L14-L18]. These actions accept a token input and repository URL. We will configure Backstage's `app-config` with a GitHub App token (or use Secrets) so scaffolder can call GitHub's API. For GitLab, the `gitlab:` actions (e.g. `gitlab:repo:push` and `publish:gitlab:merge-request`) operate similarly.
- **Pull Request Automation:** The scaffolder can either push to an existing repo (branch) or create a new repo (action `github:repo:create`). In our case, we use an existing "gitops-config" repo. After pushing, the `publish:github:pull-request` action opens a PR targeting the main branch [52†L14-L18]. For GitLab, `publish:gitlab:merge-request` does the same (similar API call). In all cases we set `update=true` so repeated scaffolds update the same PR if it exists.
- **Credential Handling:** Store the GitHub/GitLab personal token or app credentials in a secure store. For Backstage development, use environment variables (`GH_TOKEN`). In Kubernetes, we can mount these from a `ClusterSecretStore`. This matches OpenChoreo's model of storing tokens in a secret for workflows [64†L269-L276]. We ensure the token has *only* necessary scopes (e.g. `repo` for GitHub, `api` for GitLab).
- **Post-Merge Sync:** Configure the Git provider to send webhooks or rely on Flux/Argo polling. FluxCD can watch a branch by polling or via notification; ArgoCD can use webhooks. (Either way, no special credential is needed for the operator beyond access to the cluster and to `gitops-config`.)

(F) FluxCD vs Argo CD Tradeoffs

Both FluxCD and ArgoCD can perform the GitOps sync; choice depends on team needs [79†L1-L4].

- **FluxCD:** Lightweight, CLI-driven, and modular. It aligns with platform-engineer workflows – e.g. CI/CD pipelines (`flux bootstrap`) manage infrastructure as code. Flux has automatic image update controllers out-of-the-box. For a one-person effort focusing on automation, FluxCD's GitCLI approach fits well. It does not include a GUI, so developers rely on Git history and logs.
- **Argo CD:** Includes a rich UI and RBAC projects, making it easier for developers to visualize application states and for operators to control multi-cluster setups. If teams need an out-of-the-box

dashboard or integration with SSO, ArgoCD is advantageous. It also supports ApplicationSets for templating.

- **Key Differences (from community guides):** “Flux suits platform teams wanting modular controllers, native image automation, and a CLI-first workflow, while Argo CD suits teams needing a built-in UI, AppProject RBAC, and central multi-cluster management” [79†L1-L4] . In other words, choose FluxCD for a streamlined, code-centric stack; choose ArgoCD if developer self-service via UI is a priority.
- **Hybrid/Multi-Cluster:** Both can coexist on one cluster but should split responsibilities (e.g. Flux for core infrastructure, ArgoCD for apps) to avoid conflicts [45†L388-L392] . For this project, starting with the baseline (Argo Workflows present) suggests ArgoCD might integrate well; however, FluxCD is simpler to set up with a single repo. One person could implement either, but to minimize overhead **either** is acceptable – Pilot both on dev first [45†L393-L399] .
- **Recommendation:** Given the developer-facing goal, ArgoCD’s UI may provide better visibility for early demos. But the final choice can be deferred until prototyping. In either case, we must enable “prune and self-heal only after testing” [45†L393-L399] and use encryption for any secrets (per best practice) [45†L393-L399] .

References

- OpenChoreo documentation and samples [64†L259-L267] [68†L7-L10]
 - Backstage Scaffolder actions (GitHub/GitLab) [51†L16-L24] [52†L14-L18]
 - GitOps best practices and Flux/CD comparisons [79†L1-L4] [45†L393-L399]
 - Argo CD official docs (gitops concepts) [55†L247-L256] (for repo layout example)
 - Project 702 outline and requirements (internal)
-